

MUMPS 研究文档

目录

第一章 快速编译与测试	1
1.1 下载	1
1.2 编译	1
1.3 编译 C/C++ 算例	2
1.4 算例编写	7
第二章 稀疏直接法数学基础：多波前法与超节点法	13
2.1 消元、Schur 补与填充	13
2.2 消去树与符号结构	14
2.3 多波前法	14
2.4 超节点法	16
2.5 多波前法与超节点法的对应关系	18
2.6 从分解到求解	18
第三章 MUMPS 延迟主元	19
3.1 数值主元与延迟主元	19
3.2 MUMPS 的主元判据	19
3.3 延迟对多波前结构的改变	20
第四章 MUMPS 并行策略与内存管理	22
4.1 进程、线程与数据分布	22
4.2 分析阶段的节点代价	22
4.3 Level-0 子树映射	24
4.4 普通层、节点类型与阈值	24
4.5 owner 与 Type 1 映射	26
4.6 Type 2 的 owner、候选进程与 CANDIDATES	26
4.7 PROCNODE_STEPS 的含义	27
4.8 数值分解阶段的 MPI 组织	28
4.9 延迟主元和估计偏差对调度的影响	29
4.10 求解阶段的 MPI 并行	30
4.11 MPI 进程私有内存与无冲突访问	31
4.12 贡献块申请、复用与压缩	32
4.13 内存控制与 out-of-core	32

附录 A 延迟主元推广到超节点的两种主元块分解算法	33
A.1 非对称稠密主元块的延迟 LU	33
A.2 对称稠密主元块的延迟 Bunch–Kaufman 分解	37
附录 B KJI–SAXPY 算法与块分解	41
B.1 KJI 循环次序	41
B.2 多个秩一更新的合并	42
B.3 方形分块 LU	42
B.4 主元交换与面板更新	43
B.5 延迟主元下的分块 LU	44
B.6 对称块更新	44
B.7 块三角求解	45
B.8 运算量与数据访问	45
参考资料	46

第 1 章

快速编译与测试

1.1 下载

下载软件包，并切换到自己的 Git 分支：

```
git clone https://github.com/HanHFEM/mumps_cxx.git
cd mumps_cxx
git checkout -b xxx
```

如果要把本地分支上传到远端，指定远端和分支：

```
git push origin xxx
```

1.2 编译

1.2.1 安装必需库

安装 Fortran 编译器：

```
sudo apt-get install gfortran
```

安装 OpenMPI：

```
sudo apt-get install libopenmpi-dev
```

安装 ScaLAPACK、METIS、ParMETIS 和 LAPACK：

```
sudo apt-get install libscalapack-openmpi-dev
sudo apt-get install libscalapack-mpi-dev
sudo apt-get install libmetis-dev
sudo apt-get install libparmetis-dev
sudo apt-get install liblapack-dev
```

不同 Ubuntu 版本提供的 ScaLAPACK 包可能不同。若两个 ScaLAPACK 包互相冲突，应根据当前 MPI 实现选择其中一个。

1.2.2 配置附加选项

进入 MUMPS 目录：

```
cd mumps-5.6.2.2
```

可以参考项目中的以下说明文件：

- Readme_options.md;
- Readme_ordering.md;
- Readme_LAPCK.md。

默认使用的配置命令为:

```
cmake -B build \  
-Dparmetis=yes \  
-DCMAKE_INSTALL_PREFIX=./install \  
-DCMAKE_PREFIX_PATH=/mingw64 \  
-Dparallel=true
```

参数含义:

- -Dparmetis=yes: 启用 ParMETIS 等相关功能;
- -DCMAKE_INSTALL_PREFIX=./install: 把安装目录设为当前目录下的 install;
- -DCMAKE_PREFIX_PATH=/mingw64: 增加依赖库搜索前缀, 该路径需要根据本机环境调整;
- -Dparallel=true: 启用 MPI 并行版本。

1.2.3 执行构建

```
cmake --build build
```

进入 mumps-5.6.2.2/build 后, 可以看到典型构建产物:

```
libdmumps.a  
libmumps_common.a  
libpord.a  
libsmumps.a  
以及一系列.mod文件
```

1.3 编译 C/C++ 算例

1.3.1 修改测试目录的 CMakeLists.txt

在 mumps_cxx/mumps-5.6.2.2/test 中增加:

- compile_test.c;
- cpp_example.cpp。

修改后的 test/CMakeLists.txt 整理如下:

1.3.1.1 项目语言

```
project(MUMPS_Example LANGUAGES C CXX Fortran)
```

定义项目名称，并声明项目使用 C、C++ 和 Fortran。

1.3.1.2 MPI 和 hwloc 路径

```
link_directories(/usr/lib/x86_64-linux-gnu)
set(MPI_INCLUDE_DIR /usr/lib/x86_64-linux-gnu/openmpi/include)
set(MPI_C_LIBRARY /usr/lib/x86_64-linux-gnu/openmpi/lib/libmpi.so)
set(MPI_CXX_LIBRARY /usr/lib/x86_64-linux-gnu/openmpi/lib/libmpi_cxx.so)

set(HWLOC_LIBRARY /usr/lib/x86_64-linux-gnu/libhwloc.so)
set(HWLOC_INCLUDE_DIR /usr/include)
```

如果 CMake 因为环境或版本问题找不到某些库，可以显式指定路径。配置顶层工程时也要保证这些依赖路径可见。

```
include_directories(${MPI_INCLUDE_DIR} ${HWLOC_INCLUDE_DIR})
```

这条命令把 MPI 和 hwloc 头文件目录加入编译器搜索路径。

1.3.1.3 mumps_test 函数

```
set_property(DIRECTORY PROPERTY LABELS "unit;mumps")

function(mumps_test name tgt)
  if(parallel)
    add_test(
      NAME ${name}
      COMMAND ${MPIEXEC_EXECUTABLE} ${MPIEXEC_NUMPROC_FLAG} 2 $<TARGET_FILE:${tgt}>
    )
  else()
    add_test(NAME ${name} COMMAND ${tgt})
  endif()
endfunction()
```

mumps_test 接受两个参数：

- name: CTest 中的测试名称；
- tgt: 需要运行的 CMake 可执行目标。

当 parallel 为真时，测试命令包含：

- \${MPIEXEC_EXECUTABLE}: MPI 启动程序；
- \${MPIEXEC_NUMPROC_FLAG}: 指定进程数的参数；
- 2: 启动两个 MPI 进程；
- \$<TARGET_FILE:\${tgt}>: 目标可执行文件的完整路径。

当 parallel 为假时，CTest 直接执行目标程序。

1.3.1.4 双精度配置测试

```
if(BUILD_DOUBLE)
  add_executable(mumpscfg test_mumps.f90)
  target_link_libraries(mumpscfg PRIVATE MUMPS::MUMPS)
  mumps_test(Cfg mumpscfg)
endif()
```

当启用双精度实数版本时：

1. 从 test_mumps.f90 创建 mumpscfg;
2. 将它链接到 MUMPS::MUMPS;
3. 注册名为 Cfg 的测试。

1.3.1.5 版本检查

```
if(MUMPS_UPSTREAM_VERSION VERSION_LESS 5.1)
  return()
endif()
```

如果上游 MUMPS 版本低于 5.1，则停止处理当前 CMakeLists。

1.3.1.6 单精度和双精度 Fortran 算例

```
if(BUILD_SINGLE)
  add_executable(s_simple s_simple.f90)
  target_link_libraries(s_simple PRIVATE MUMPS::MUMPS)
  mumps_test(SimpleReal32 s_simple)
endif()

if(BUILD_DOUBLE)
  add_executable(d_simple d_simple.f90)
  target_link_libraries(d_simple PRIVATE MUMPS::MUMPS)
  mumps_test(SimpleReal64 d_simple)
endif()
```

- s_simple 对应单精度实数版本;
- d_simple 对应双精度实数版本。

1.3.1.7 C 算例

```
add_executable(Csimple simple.c)
target_link_libraries(Csimple PRIVATE MUMPS::MUMPS)
mumps_test(CsimpleReal64 Csimple)
```

这三行从 simple.c 创建 Csimple，链接 MUMPS，并注册一个双精度实数 C 接口测试。

1.3.1.8 新增 C/C++ 算例

```
add_executable(compile_test compile_test.c)
target_link_libraries(compile_test PRIVATE MUMPS::MUMPS)

add_executable(cpp_example cpp_example.cpp)
target_link_libraries(cpp_example PRIVATE
    ${HWLOC_LIBRARY}
    ${MPI_C_LIBRARY}
    ${MPI_CXX_LIBRARY}
    MUMPS::MUMPS
)
```

C++ 程序显式链接了 hwloc、MPI C 库、MPI C++ 库和 MUMPS。

1.3.1.9 测试属性和 Windows DLL

```
get_property(test_names DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR} PROPERTY TESTS)
set_property(TEST ${test_names} PROPERTY RESOURCE_LOCK cpu_mpi)
set_property(TEST ${test_names} PROPERTY TIMEOUT 30)
```

- get_property 取得当前目录注册的全部测试；
- RESOURCE_LOCK cpu_mpi 防止多个测试同时占用同一 MPI 测试资源；
- TIMEOUT 30 把超时时间设为 30 秒。

```
if(WIN32 AND BUILD_SHARED_LIBS)
    set_property(TEST ${test_names} PROPERTY
        ENVIRONMENT_MODIFICATION
        "PATH=path_list_append:${CMAKE_INSTALL_FULL_BINDIR};PATH=path_list_append:${CMAKE_PREFIX_PATH}/bin;PATH=path_list_append:${PROJECT_BINARY_DIR}"
    )
endif()
```

这段只在 Windows 共享库构建中生效，用于把 DLL 所在目录加入测试进程的 PATH。

构建完成后，需要在 build 目录中找到生成的可执行文件，或者使用 ctest 运行已注册测试。

1.3.2 新建一个测试目录

1.3.2.1 继续使用 CTest 框架

在 mumps_cxx 下新建目录，放入一个 C 源文件和一个简化的 CMakeLists.txt:

```
set_property(DIRECTORY PROPERTY LABELS "unit;mumps")

function(mumps_test name tgt)
    if(parallel)
```



```

    add_test(
      NAME ${name}
      COMMAND ${MPIEXEC_EXECUTABLE} ${MPIEXEC_NUMPROC_FLAG} 2 $<TARGET_FILE:${tgt}>
    )
  else()
    add_test(NAME ${name} COMMAND ${tgt})
  endif()
endfunction()

if(MUMPS_UPSTREAM_VERSION VERSION_LESS 5.1)
  return()
endif()

if(BUILD_DOUBLE)
  add_executable(dotest testano.c)
  target_link_libraries(dotest PRIVATE MUMPS::MUMPS)
  mumps_test(doTEST dotest)
endif()

get_property(test_names DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR} PROPERTY TESTS)
set_property(TEST ${test_names} PROPERTY RESOURCE_LOCK cpu_mpi)
set_property(TEST ${test_names} PROPERTY TIMEOUT 30)

```

还要在顶层 CMakeLists.txt 中加入该子目录：

```

include(cmake/summary.cmake)
include(cmake/install.cmake)

add_subdirectory(coderun)

file(GENERATE OUTPUT .gitignore CONTENT "*")

```

重新构建后，可使用 `ctest` 运行新程序。

1.3.2.2 构建完成后直接运行

如果不想使用 CTest，可以为目标添加构建后命令：

```

set_property(DIRECTORY PROPERTY LABELS "unit;mumps")

if(MUMPS_UPSTREAM_VERSION VERSION_LESS 5.1)
  return()
endif()

if(BUILD_DOUBLE)
  add_executable(dotest testano.c)
  target_link_libraries(dotest PRIVATE MUMPS::MUMPS)

  add_custom_command(

```

```

    TARGET dotest POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E echo "Running dotest..."
    COMMAND ${TARGET_FILE:dotest}
    DEPENDS dotest
)
endif()

```

这样执行 `cmake --build build` 时，不仅会构建 `dotest`，还会在构建完成后立即运行它。

1.3.2.3 数值类型构建选项

`mumps-5.6.2.2/options.cmake` 中包含：

```

option(BUILD_SINGLE "Build single precision float32 real" ON)
option(BUILD_DOUBLE "Build double precision float64 real" ON)
option(BUILD_COMPLEX "Build single precision complex")
option(BUILD_COMPLEX16 "Build double precision complex")

```

这些选项分别控制：

选项	数值类型
<code>BUILD_SINGLE</code>	单精度实数
<code>BUILD_DOUBLE</code>	双精度实数
<code>BUILD_COMPLEX</code>	单精度复数
<code>BUILD_COMPLEX16</code>	双精度复数

配置中启用了单精度和双精度实数版本，没有启用复数版本。

1.4 算例编写

1.4.1 `simple.c` 研究

`test/simple.c` 使用 DMUMPS 的 C 接口求解：

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 4 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

完整代码如下：

1.4.1.1 头文件

```

#include <stdio.h>
#include <string.h>

```

```
#include "mpi.h"
#include "dmumps_c.h"
```

dmumps_c.h 位于:

```
mumps-5.6.2.2/build/_deps/mumps-src/include/dmumps_c.h
```

它定义了双精度实数 MUMPS 的 C 接口, 包括:

- DMUMPS_STRUC_C 参数结构体;
- dmumps_c() 入口函数。

```
void MUMPS_CALL dmumps_c(DMUMPS_STRUC_C *dmumps_par);
```

1.4.1.2 宏常量

```
#define JOB_INIT -1
#define JOB_END -2
#define USE_COMM_WORLD -987654
```

- JOB_INIT: 初始化 MUMPS 实例;
- JOB_END: 终止 MUMPS 实例;
- USE_COMM_WORLD: 使用 MPI_COMM_WORLD。

1.4.1.3 矩阵和右端项

```
DMUMPS_STRUC_C id;
MUMPS_INT n = 2;
MUMPS_INT8 nnz = 2;
MUMPS_INT irn[] = {1, 2};
MUMPS_INT jcn[] = {1, 2};
double a[2];
double rhs[2];
```

- n: 矩阵阶数;
- nnz: 非零元数量;
- irn/jcn: 非零元的 1-based 全局行列索引;
- a: 非零元数值;
- rhs: 右端项, 求解完成后集中式解也写回该数组。

MUMPS_INT8 表示 MUMPS 使用的 64 位整数类型, 不是 8 位整数。启用 -DINTSIZE64 时, MUMPS_INT 也会变成 64 位; 某些 MPI C 接口参数仍要求标准 C int。

1.4.1.4 初始化 MPI 和 MUMPS

```

ierr = MPI_Init(&argc, &argv);
ierr = MPI_Comm_rank(MPI_COMM_WORLD, &myid);

id.comm_fortran = USE_COMM_WORLD;
id.par = 1;
id.sym = 0;
id.job = JOB_INIT;
dmumps_c(&id);

```

- MPI_Init 初始化 MPI 环境;
- MPI_Comm_rank 取得当前 rank;
- comm_fortran 指定 MUMPS 通信器;
- par=1 表示 host 也参与分解和求解;
- sym=0 选择非对称路径。虽然示例矩阵实际为对称对角阵，选择非对称路径仍可求解;
- job=-1 初始化 MUMPS 实例。

1.4.1.5 只在 host 上提供集中式矩阵

```

if (myid == 0) {
    id.n = n;
    id.nnz = nnz;
    id.irn = irn;
    id.jcn = jcn;
    id.a = a;
    id.rhs = rhs;
}

```

默认集中式输入模式下，rank 0 提供矩阵和右端项，其他 rank 参与 MUMPS 内部并行计算。

1.4.1.6 C 数组下标与手册编号

```

#define ICNTL(I) icntl[(I)-1]

```

手册中的控制参数从 1 开始编号，而 C 数组从 0 开始。该宏使代码可以继续使用手册中的编号，例如 id.ICNTL(1) 实际访问 id.icntl[0]。

```

id.ICNTL(1) = -1;
id.ICNTL(2) = -1;
id.ICNTL(3) = -1;
id.ICNTL(4) = 0;

```

示例关闭了主要输出信息。

1.4.1.7 分析、分解和求解

```
id.job = 6;
dmumps_c(&id);
```

JOB=6 等价于依次执行：

1. 分析；
2. 数值分解；
3. 求解。

1.4.1.8 错误检查和释放

```
if (id.infog[0] < 0) {
    fprintf(stderr,
        " (PROC %d) ERROR RETURN: INFOG(1)= %d INFOG(2)= %d\n",
        myid, id.infog[0], id.infog[1]);
    return 1;
}

id.job = JOB_END;
dmumps_c(&id);
MPI_Finalize();
```

INFOG(1)<0 表示调用失败，INFOG(2) 通常提供附加错误信息。程序结束前先以 JOB=-2 释放 MUMPS 实例，再调用 MPI_Finalize。

1.4.2 JOB 参数

在调用 MUMPS 前，所有 MPI 进程都要设置相同的 JOB。MUMPS 不会修改该字段。

JOB	作用
-1	初始化；调用前先设置 comm_fortran、sym 和 par
-2	终止实例并释放内部资源
1	分析矩阵结构
2	数值分解，要求已有成功的分析结果
3	求解，要求已有成功的数值分解
4	分析和数值分解
5	数值分解和求解
6	分析、数值分解和求解
7	把 MUMPS 内部实例保存到磁盘
-3	删除保存到磁盘的实例数据
8	从磁盘恢复 MUMPS 实例
-4	释放数值阶段数据，但保留分析阶段结构
9	在求解前计算分布式右端项的可能分布

JOB=9 只能在成功完成分解，即 JOB=2/4 返回且 INFOG(1)>=0 后使用。

1.4.3 PAR 参数

PAR 的取值为：

- PAR=0: host 不参加分解和求解的并行计算，主要负责初始问题、分析协调、数据分发和结果收集；
- PAR=1: host 也参加分解和求解；其他值按 1 处理。

所有进程应提供相同的 PAR。如果不一致，MUMPS 使用 rank 0 上的值。

当矩阵集中在 rank 0 且输入规模很大时，PAR=1 可能让 rank 0 同时承担原始矩阵和数值工作区，从而造成内存不均衡。

1.4.4 矩阵输入格式

详细接口可以参考 MUMPS Users' Guide 第 5.2 节。

1.4.4.1 三类输入结构

```
/* Assembled entry */
MUMPS_INT      nz;
MUMPS_INT8     nnz;
MUMPS_INT      *irn;
MUMPS_INT      *jcn;
DMUMPS_COMPLEX *a;

/* Distributed entry */
MUMPS_INT      nz_loc;
MUMPS_INT8     nnz_loc;
MUMPS_INT      *irn_loc;
MUMPS_INT      *jcn_loc;
DMUMPS_COMPLEX *a_loc;

/* Element entry */
MUMPS_INT      nelt;
MUMPS_INT      *eltptr;
MUMPS_INT      *eltvar;
DMUMPS_COMPLEX *a_elt;
```

1.4.4.2 SYM

SYM 指定矩阵类型：

- 0: 非对称矩阵；
- 1: 对称正定矩阵，使用不进行数值主元搜索的正定路径；
- 2: 一般对称矩阵；

- 其他值按 0 处理。

所有进程应提供相同的 `SYM`, 否则使用 rank 0 上的值。MUMPS 不提供专门的 Hermitian 矩阵模式。

第 2 章

稀疏直接法数学基础：多波前法与超节点法

本章说明多波前法和超节点法的数学结构。多波前法以消去树节点对应的波前矩阵为计算单位，超节点法把非零结构相同或相近的一组连续列作为计算单位。两者都将标量稀疏消元重组为稠密块运算，但在数据组织和更新传播方式上有所区别 [6, 8, 7]。

2.1 消元、Schur 补与填充

考虑稀疏线性系统

$$Ax = b, \quad A \in \mathbb{R}^{n \times n}.$$

一般非对称分解写为

$$PAQ = LU,$$

其中 P, Q 是行、列置换矩阵。对称分解写为

$$P^T A P = LDL^T.$$

对称正定情形可以写成 Cholesky 形式 $P^T A P = LL^T$ ；一般对称不定情形的 D 由 1×1 和 2×2 对角块组成。

把矩阵分成主元变量 p 和其余变量 r ：

$$A = \begin{bmatrix} A_{pp} & A_{pr} \\ A_{rp} & A_{rr} \end{bmatrix}.$$

若 A_{pp} 可逆，则

$$A = \begin{bmatrix} I & 0 \\ A_{rp}A_{pp}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{pp} & A_{pr} \\ 0 & S \end{bmatrix}, \quad S = A_{rr} - A_{rp}A_{pp}^{-1}A_{pr}.$$

S 称为 Schur 补。原来为零的位置可能在 Schur 补中变成非零，这些新非零称为填充。稀疏排序通过改变变量消去次序控制填充、因子非零元数和中间稠密块的规模。

对称矩阵的非零结构可以表示为无向图。消去一个顶点时，该顶点尚未消去的邻接点形成团；为形成这个团而增加的边对应填充。非对称矩阵通常通过对称化结构或二部图结构进行符号分析。

2.2 消去树与符号结构

消去树记录消元更新的依赖关系。以对称情形为例，若 L 是符号 Cholesky 因子，则列 j 的父节点可以定义为 $L(:, j)$ 中行号大于 j 的第一个非零位置。子节点消元产生的 Schur 补更新沿父指针向上传递。

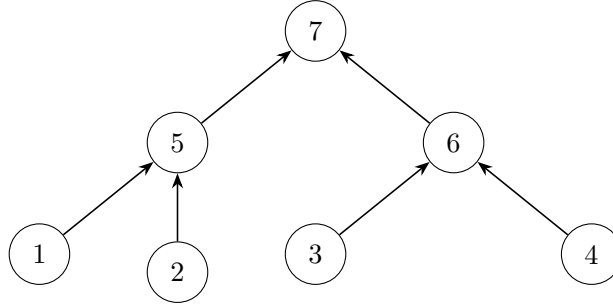


图 2.1 消去树中的更新方向：子节点指向父节点。

消去树同时给出两种顺序：数值分解按后序从叶到根处理；使用因子求解时，前代和回代分别沿因子依赖的两个方向进行。位于不同树枝且依赖已经满足的节点彼此独立。

排序、符号分解和数值分解的作用不同：排序确定结构次序；符号分解预测因子非零结构和树依赖；数值分解计算实际因子，并根据数值主元选择修正符号预测。

2.3 多波前法

2.3.1 波前矩阵

对消去树中的一个节点，把本节点可以消去的变量记为 E ，仍需传递到祖先节点的更新变量记为 U 。按照 E, U 排列，节点的稠密波前矩阵为

$$F = \begin{bmatrix} F_{EE} & F_{EU} \\ F_{UE} & F_{UU} \end{bmatrix}.$$

E 中的变量称为完全求和（fully summed）变量：与这些变量有关的原矩阵项和所有后代更新均已装配到当前波前，因此它们是当前节点的候选消去变量。 U 中的信息尚未在当前节点完全求和，只参与更新。

非对称情形消去 E 后得到

$$P_F F Q_F = \begin{bmatrix} L_{EE} & 0 \\ L_{UE} & I \end{bmatrix} \begin{bmatrix} U_{EE} & U_{EU} \\ 0 & C \end{bmatrix},$$

其中贡献块为

$$C = F_{UU} - F_{UE} F_{EE}^{-1} F_{EU}.$$

对称情形相应地写成

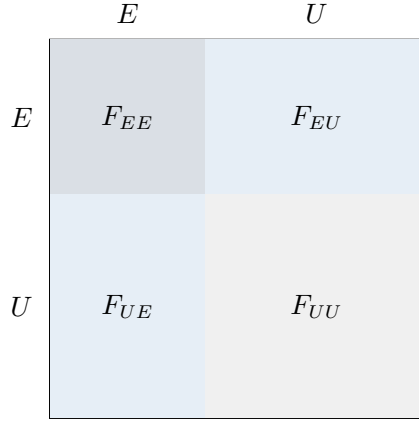


图 2.2 方形波前矩阵的主元区和更新区。

$$P_F^T F P_F = \begin{bmatrix} L_{EE} & 0 \\ L_{UE} & I \end{bmatrix} \begin{bmatrix} D_{EE} & 0 \\ 0 & C \end{bmatrix} \begin{bmatrix} L_{EE}^T & L_{UE}^T \\ 0 & I \end{bmatrix},$$

$$C = F_{UU} - L_{UE} D_{EE} L_{UE}^T.$$

节点保存已经得到的因子块，并把 C 传给父节点。

2.3.2 装配与 extend-add

当前波前由原矩阵项和子节点贡献块相加得到。设 $F^{(A)}$ 是装配到本节点的原矩阵项，子节点 c 的贡献块为 C_c ， R_c 把子节点局部变量编号映射到当前波前局部编号，则

$$F = F^{(A)} + \sum_{c \in \text{child}(F)} R_c^T C_c R_c.$$

该操作称为扩展累加 (extend-add)。 R_c 不要求子节点变量在父波前中连续；它根据全局变量编号完成嵌入和累加。所有子贡献到达后，本节点的完全求和区才完整。

叶节点只装配原矩阵项；内部节点同时装配原矩阵项和子贡献；根节点消去剩余变量，不再向父节点产生贡献块。因而，多波前数值分解可以表示成消去树上的递归：

$$\text{factor}(v) : \quad \{\text{factor}(c)\}_{c \in \text{child}(v)} \longrightarrow \text{assemble}(F_v) \longrightarrow \text{eliminate}(F_v) \longrightarrow C_v.$$

2.3.3 波前规模与运算量

设一个波前有 f 个总变量，其中 p 个在本节点消去，更新区规模为

$$u = f - p.$$

非对称情形需要存储主元列和主元行；其数量级为

$$p(2f - p).$$

对称情形只需存储一个三角部分，对应数量级为

$$pf.$$

数值工作由主元块分解、块行/块列求解和 Schur 补更新组成。当 $f \gg p$ 时，主要计算来自 $p \times u$ 与 $u \times p$ 形成的尾部更新；当 p 较大时，主元块自身的稠密分解也占据主要部分。

2.4 超节点法

2.4.1 超节点的定义

以列式因子 L 为例，若一组连续列

$$J = \{s, s + 1, \dots, t\}$$

在对角块以下具有相同的非零行结构，则这些列可以组成一个超节点。设公共行集为 R ，超节点在 L 中的数值部分写成

$$L_{J \cup R, J} = \begin{bmatrix} L_{JJ} \\ L_{RJ} \end{bmatrix},$$

其中 L_{JJ} 是稠密下三角块， L_{RJ} 是稠密矩形块。

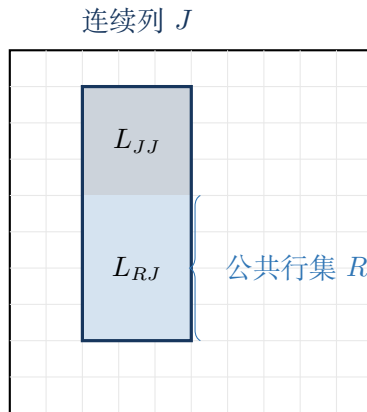


图 2.3 超节点的连续列及其公共非零行结构。

严格超节点要求列结构完全一致。实际算法还可以把结构接近的相邻列合并为松弛超节点，合并后显式保存由此产生的少量结构零。基本超节点由符号因子的精确结构确定；松弛合并基本超节点之上进行 [7]。

2.4.2 超节点分解公式

在 Cholesky 或 LDL^T 分解中，设前面超节点对当前列块 J 的更新已经累加，当前块写成

$$\begin{bmatrix} A_{JJ}^{(k)} \\ A_{RJ}^{(k)} \end{bmatrix}.$$

Cholesky 情形先计算

$$L_{JJ}L_{JJ}^T = A_{JJ}^{(k)},$$

再进行三角求解

$$L_{RJ} = A_{RJ}^{(k)}L_{JJ}^{-T},$$

随后对尚未分解部分施加块更新

$$A_{RR}^{(k+1)} = A_{RR}^{(k)} - L_{RJ}L_{RJ}^T.$$

一般对称不定情形写为

$$A_{JJ}^{(k)} = L_{JJ}D_{JJ}L_{JJ}^T,$$

$$L_{RJ} = A_{RJ}^{(k)}L_{JJ}^{-T}D_{JJ}^{-1},$$

$$A_{RR}^{(k+1)} = A_{RR}^{(k)} - L_{RJ}D_{JJ}L_{RJ}^T.$$

一般非对称超节点 LU 使用行超节点和列超节点形成面板。其块公式为

$$\begin{bmatrix} A_{JJ} & A_{JK} \\ A_{KJ} & A_{KK} \end{bmatrix} = \begin{bmatrix} L_{JJ} & 0 \\ L_{KJ} & I \end{bmatrix} \begin{bmatrix} U_{JJ} & U_{JK} \\ 0 & S_{KK} \end{bmatrix},$$

$$U_{JK} = L_{JJ}^{-1}A_{JK}, \quad L_{KJ} = A_{KJ}U_{JJ}^{-1},$$

$$S_{KK} = A_{KK} - L_{KJ}U_{JK}.$$

这些公式把一组列的标量更新合并为三角求解和矩阵乘法。

2.4.3 左看、右看与上看组织

超节点分解可以按更新的产生和使用次序分类。

- 左看：处理当前超节点时，收集所有以前超节点对它的更新，然后分解当前块；
- 右看：一个超节点分解完成后，立即把它的更新散布到所有后续相关超节点；
- 上看：按当前列或列块遍历因子结构，沿依赖索引寻找并累加以前列块的贡献。

三种组织使用相同的块分解恒等式，区别在于 Schur 补更新何时形成、存放在哪里以及何时累加到目标列块。

2.5 多波前法与超节点法的对应关系

两类方法都建立在排序后的消元结构上，也都用连续或相关的变量集合形成稠密块。它们的主要对应关系见表 2.1。

表 2.1 多波前法与超节点法的数学组织

项目	多波前法	超节点法
基本单位	消去树节点的波前矩阵	具有共同列结构的连续列块
子问题输入	原矩阵项和子节点贡献块	以前超节点对当前列块的更新
中间结果	显式贡献块 C	对后续超节点的块更新
结构索引	波前变量列表和父子关系	超节点列区间及其行结构
稠密计算	波前主元块分解与 Schur 补更新	对角块分解、TRSM 和块更新
依赖表示	波前消去树	超节点消去树或列依赖图

若把一个超节点及其尚未消去的更新行装配成方形局部矩阵，就得到与波前矩阵相同的 E/U 分块；若把一个波前中连续且结构一致的主元列作为列块，则其主元计算又具有超节点形式。因此，两者的 Schur 补代数相同，差别主要在于稀疏数据如何分组和更新如何组织。

2.6 从分解到求解

分解得到因子后，一般非对称系统按

$$y = L^{-1}Pb, \quad x = QU^{-1}y$$

求解；对称系统按

$$y = L^{-1}P^Tb, \quad z = D^{-1}y, \quad x = PL^{-T}z$$

求解。多波前表示按消去树节点访问因子块，超节点表示按超节点列块访问因子。多个右端项时，标量或向量三角更新变成块三角求解和矩阵乘法。

第 3 章

MUMPS 延迟主元

3.1 数值主元与延迟主元

符号分析根据非零结构预测节点 v 的候选消去变量数 p_v 和波前规模 f_v 。数值分解时，只有满足稳定性判据的完全求和（fully summed）变量才能在当前波前中消去。若实际接受的主元数为 $\hat{p}_v < p_v$ ，则其余

$$d_v = p_v - \hat{p}_v$$

个变量被保留在贡献块中并传给父节点，这些变量称为延迟主元或延迟变量。延迟主元是多波前法处理符号预测与数值稳定性不一致的基本机制 [3, 1]。

设当前波前按候选消去区 E 和更新区 U 分块。若 E 中一部分变量 D 被延迟，其余已接受主元记为 A ，则分解后传递的变量集合为

$$U' = U \cup D.$$

贡献块的阶数从符号预测的 $|U|$ 增加为

$$|U'| = |U| + |D|.$$

父节点装配该贡献块时，延迟变量成为父波前中的完全求和候选变量，并重新进行主元测试。若仍不能接受，可以继续向更高祖先传递。

3.2 MUMPS 的主元判据

3.2.1 非对称阈值部分选主元

一般非对称分解采用阈值部分选主元。候选主元满足

$$|a_{ii}| \geq u \max_{k=i, \dots, n} |a_{ki}|, \quad 0 \leq u \leq 1,$$

其中 u 由 CNTL(1) 控制。只有完全求和区中的变量可以作为当前波前的普通候选主元。 $u = 1$ 对应列部分选主元，较小的 u 允许接受相对较小的候选主元；相应单步增长因子的上界为 $1 + 1/u[1]$ 。

阈值检验失败时，MUMPS 不在当前节点消去该候选变量，而是把它并入贡献块。该行为与普通稠密 LU 在同一个固定矩阵上继续搜索不同，因为多波前法还受到完全求和集合和消去树依赖的限制。

3.2.2 对称不定主元

一般对称不定分解采用

$$P^T A P = L D L^T,$$

其中 D 包含 1×1 与 2×2 主元块。主元选择同时比较对角元素和相关列、行的非对角元素；可接受的 1×1 或 2×2 块在当前波前中消去，不能接受的变量进入延迟集合。对称交换必须同时作用于行和列，最终只保留一个对称置换 P 。

3.2.3 静态主元

静态主元处理由 CNTL(4) 控制。静态主元方法对过小或零主元进行数值扰动，使预定主元可以继续使用，并可与 CNTL(1) 的阈值判据组合。采用静态主元时通常配合缩放，并在求解后执行迭代改进，以减小扰动造成的误差 [1]。

延迟主元与静态主元是两种不同处理：延迟主元改变变量在消去树上的实际消去位置；静态主元保留既定位置并修改用于分解的主元数值。

3.3 延迟对多波前结构的改变

设当前节点原计划消去 p 个变量，波前阶数为 f ，更新区阶数为 $u = f - p$ 。发生 d 个延迟后，本节点实际消去 $p - d$ 个变量，贡献块阶数变为

$$u + d.$$

因此延迟会改变以下实际量：

1. 本节点保存的因子列数由 p 变为 $p - d$ ；
2. 本节点贡献块的行列数增加；
3. 父节点装配时增加 d 个变量及其数值项；
4. 父节点及其祖先的实际波前、运算量和因子存储量可能超过分析估计；
5. 贡献块的通信长度和进程私有工作空间需求随之增加。

延迟不会重做符号排序，也不会重新构造一棵新的静态消去树。变量沿原有父链向上传递，数值分解在预先建立的树依赖和进程映射内处理实际增大的波前。

3.3.1 运行时计数与贡献块扩张

在数值分解中，NASS 表示当前波前原计划尝试消去的完全求和变量数，NPIV 表示实际通过主元检验的变量数。节点结束时的延迟量为

$$NELIM = NASS - NPIV.$$

贡献块由两类变量共同组成：

$$\underbrace{NASS - NPIV}_{\text{延迟变量}} + \underbrace{NFRONT - NASS}_{\text{普通更新变量}}.$$

因此，延迟变量不是存入一个与贡献块分离的全局队列，而是随贡献块及其变量索引一起传给父节点。父节点装配子贡献时累加实际延迟量，并据此扩大自己的波前。该处理由数值分解后的实际 $NPIV$ 决定，而不是在符号分析阶段预先确定 [2]。

第 4 章

MUMPS 并行策略与内存管理

MUMPS 采用异步多波前并行方法。分析阶段建立消去树映射、节点 owner 和大型节点的候选进程集合；数值分解阶段依据依赖满足情况、消息到达顺序和受限的运行时负载信息调度任务。因此，其并行组织是静态映射与动态任务调度的结合，而不是完全静态的层同步，也不是允许任意进程接管任意节点的全局工作窃取 [1, 4, 5]。

4.1 进程、线程与数据分布

4.1.1 MPI 工作进程

MUMPS 初始化时已经知道 MPI 通信域大小和 PAR。若 PAR=1，host rank 同时是工作进程；若 PAR=0，host 不承担数值工作，其余 rank 组成工作通信域。内部量 NSLAVES 或 SLAVEF 表示工作进程数。该数量在区分 Type 1 和 Type 2 节点之前已经确定，因此节点分类不参与决定 MPI 进程总数。

master、slave、owner 和 candidate 都是 MPI rank 在特定任务中的角色，不等同于物理计算节点。一台机器上可以运行多个 MPI rank，一个 rank 在不同波前中也可以承担不同角色。

4.1.2 三类并行来源

MUMPS 中可以同时出现三类并行：

1. 消去树并行：不同 MPI 进程同时处理不同树枝上的波前；
2. 波前内部 MPI 并行：一个大型 Type 2 波前由一个 master 和若干参与进程协同处理；
3. MPI 进程内部线程并行：多线程 BLAS 以及 MUMPS 的部分 OpenMP 区域在同一 rank 的共享地址空间中执行。

第一、二项是 MPI 进程间并行，第三项是进程内共享内存并行。混合版本的线程并行主要来自多线程 BLAS，同时包含若干由 OpenMP 实现的并行区域 [1]。

4.2 分析阶段的节点代价

4.2.1 单个波前的规模

在 mumps_static_mapping.F 中，MUMPS_TREECOSTS 对节点 pos 取得

$$p = \text{npiv}, \quad f = \text{nfront},$$

其中 p 是节点计划消去的变量数, f 是波前总阶数, 贡献块的预测阶数为

$$c = f - p.$$

调用

```
CALL MUMPS_CALCNODECOSTS(
    npiv,
    nfront,
    cv_ncostw(pos),
    cv_ncostm(pos))
```

后, `cv_ncostw(pos)` 保存该节点的计算量估计, `cv_ncostm(pos)` 保存该节点的因子存储量估计。这里的“计算量”是稠密主元块分解、三角求解和 Schur 补更新的浮点运算估计; “因子存储量”是该节点预计保留的因子数值个数, 不是进程已经申请的字节数。

MUMPS 5.6.2 的普通非低秩路径使用以下公式。一般非对称矩阵为

$$W_v = 2fp(f - p - 1) + \frac{p(p+1)(2p+1)}{3} + \frac{(2f - p - 1)p}{2},$$

$$M_v = p(2f - p).$$

对称矩阵为

$$W_v = p \left[f^2 + 2f - (f+1)(p+1) + \frac{(p+1)(2p+1)}{6} \right],$$

$$M_v = pf.$$

实现将 W_v, M_v 分别存入 `cv_ncostw` 和 `cv_ncostm`。这两个数用于分析阶段的相对负载比较和节点映射 [2]。

4.2.2 子树累计代价

MUMPS_TREECOSTS 递归遍历实际子节点, 定义

$$T_v^W = W_v + \sum_{c \in \text{child}(v)} T_c^W, \quad T_v^M = M_v + \sum_{c \in \text{child}(v)} T_c^M.$$

对应数组为 `cv_tcostw(v)` 和 `cv_tcostm(v)`。单节点代价用于普通层中的节点映射; 累计代价用于把一个完整的 Level-0 子树作为整体映射。分析阶段使用这些估计更新每个进程的累计工作量

`cv_proc_workload(proc)`

和累计内存量

`cv_proc_memused(proc).`

4.3 Level-0 子树映射

Level-0 不是“树深度等于 0 的所有节点”，也不是“只由一个进程执行的一棵唯一子树”。它是消去树底部的一组互不重叠子树。每棵 Level-0 子树有一个边界根；该根以下的全部后代属于同一子树。

MUMPS_ROOTLIST 先从消去树根列表开始，MUMPS_LAYERLO 再把当前边界逐步向叶方向展开，形成数量足够、代价可分配的树底子树。因此，构造过程从根列表开始细化，而最终得到的是靠近叶侧的子树划分。

标准路径中的 Level-0 最小候选数量为

$$\text{MINSIZE_LO} = 3 \text{ SLAVEF}.$$

内部控制路径还可以取 6 SLAVEF 或 2 SLAVEF。这些值控制映射阶段希望形成的 Level-0 边界规模，不表示每个进程只能获得一棵子树。

MUMPS_ARRANGELO 按子树累计代价调用 MUMPS_FIND_BEST_PROC:

$$\text{Level-0 根 } r \mapsto \underset{q \in \mathcal{P}}{\operatorname{argmin}} \{ \text{当前累计负载或内存} \},$$

同时受进程可用性和内存约束限制。确定边界根的 owner 后，MUMPS_MAPSUBTREE 调用 MUMPS_MAPBELOW，递归执行

$$\text{procnode}(\text{descendant}) = \text{procnode}(\text{subtree_root}).$$

例如，若 Level-0 根 A 被映射到 rank 2，则 A 以下的 B, C, D, E 也映射到 rank 2。该子树内部的贡献块在同一 MPI 地址空间中装配；一个 rank 可以拥有多棵 Level-0 子树。

4.4 普通层、节点类型与阈值

4.4.1 layernmb

layernmb 是静态映射算法中的层编号。它不是 MPI rank，也不是树的数学深度数组。

layernmb=0 对应 Level-0 子树层；正值表示 Level-0 边界以上依次处理的映射层。

MUMPS_FIND_THIS_LAYER 产生该层的节点列表，随后执行节点分类、代价处理和进程映射。

4.4.2 Type 0 和 Type 1

多进程情况下，Level-0 边界根被标记为 Type 0，其后代在内部标记为 (-1)，最终继承同一个 owner。只有一个工作进程时，源码把所有节点设为 Type 0。

Level-0 以上不满足 Type 2 条件、且未被指定为特殊 Type 3 的节点被设为 Type 1。Type 1 节点只有一个 owner；该进程负责装配波前、执行主要分解、保存因子并产生贡献块。

4.4.3 Type 2 的精确判据

MUMPS_ASSIGN_TYPES 中，普通层节点成为 Type 2 需要同时满足：

$$f - p > \text{KEEP}(9),$$

$$\text{ICNTL}(59) = 0,$$

并且源码变量 `in.ne.0`，即该节点具有实际子节点。

MUMPS 5.6.2 的 MUMPS_ISTYPE2BYSIZE 还写有

```
((npiv > cv_keep(4)).or.(.TRUE.))
```

因此 KEEP(4) 对 Type 2 判定实际不构成限制。标准默认值为：

表 4.1 MUMPS 5.6.2 标准默认节点阈值

矩阵类别	KEEP(4)	KEEP(9)
一般非对称	32	700
对称	24	400

所以在标准默认值下，Type 2 的有效规模条件是：

$$\begin{cases} f - p > 700, & \text{一般非对称,} \\ f - p > 400, & \text{对称,} \end{cases}$$

再加上“有实际子节点”和“ICNTL(59)=0”。不满足这些条件的普通节点为 Type 1。内部选项可以修改 KEEP 值，因此上述数字对应 MUMPS 5.6.2 双精度版本的标准默认设置 [2]。

4.4.4 Type 3

Type 3 不是“消去树根”的同义词。选择过程先在所有满足 `FRERE(i)=0` 的结构根中找到预计波前最大的根，记其阶数为 `SIZEROOT`。标准自动模式下，只有同时满足以下条件才将该根标记为 Type 3：

1. 构建时启用了 ScaLAPACK；
2. `ICNTL(13)=0`，允许并行处理最后的根稠密块；
3. 工作进程数 `SLAVEF>1`；
4. `SIZEROOT>SLAVEF`；
5. `SIZEROOT>KEEP(37)`。

默认阈值为

$$\text{KEEP}(37) = \max\left(800, \left\lceil 70\sqrt{\text{NSLAVES} + 1} \right\rceil\right).$$

因此，根节点不一定是 Type 3，但 Type 3 一定是所选消去树的结构根。其 `procnode` 中仍记录协调进程，而根矩阵和数值计算随后分布到二维 BLACS/ScaLAPACK 进程网格 [2]。

4.5 owner 与 Type 1 映射

这里的“映射”是建立

消去树节点或子树 $\rightarrow$ 负责进程及允许参与的进程集合

的关系。owner 是某个节点的主要负责 rank。对 Type 1, owner 是唯一执行该节点主要数值任务并保存因子的进程；对于子节点贡献位于其他 rank 的情况，贡献块通过 MPI 发送到父节点 owner。

MUMPS_MAP_LAYER 先按当前映射目标对节点代价排序，再由 MUMPS_SORTPROCS 排列进程。若按浮点工作量平衡，则当前累计工作量小的进程靠前；若按内存平衡，则当前累计内存小的进程靠前；启用优选进程集合时，该集合排在普通进程之前。

对一个 Type 1 节点 i ，映射器依次检查排序后的进程，选择第一个满足工作量和内存上限的进程 q ：

$$cv_procnode(i) = q.$$

随后更新

$$cv_proc_workload(q) += W_i, \quad cv_proc_memused(q) += M_i.$$

这一步使用分析估计，不是对数值分解实际时间和内存的测量。

4.6 Type 2 的 owner、候选进程与 CANDIDATES

Type 2 节点既有一个 master/owner，也有若干候选工作进程。候选数量由 MUMPS 内部的块 2 进程数模型根据 SLAVEF、矩阵类别、 f 、 $f - p$ 以及相关内部控制量计算，不需要先知道节点类型来确定 MPI 总进程数。

分析结束时，所有 Type 2 及分裂链相关节点依次写入

PAR2_NODES(INIV2).

公开数组

CANDIDATES(NSLAVES+1,NB_NIV2)

的第 INIV2 列描述 PAR2_NODES(INIV2) 对应节点。若

$$\text{PAR2_NODES}(3) = 25,$$

则 $\text{INIV2}=3$ 只表示“第三列候选信息属于节点 25”，不是第三个 MPI rank，也不是第三层。

令

$$n_c = \text{CANDIDATES}(\text{NSLAVES}+1, \text{INIV2}).$$

前 n_c 项保存主要候选进程的 0-based 工作通信域 rank。例：

$$\text{CANDIDATES}(:, 3) = [0, 2, 5, -1, \dots, 3]^T$$

表示节点 25 的主要候选进程为 rank 0、2、5，最后一项 3 是候选数量。对 Type 2 分裂链，主要候选项之后还可以保存额外参与者；负值标记该扩展列表的结束，解释时需要结合节点的分裂类型。

MUMPS_RETURN_CANDIDATES 把内部候选表复制到 $\text{id}\%\text{CANDIDATES}$ 。dana_driver.F 随后广播 PAR2_NODES 和 CANDIDATES，并在每个 rank 上建立“本 rank 是否为候选”的辅助信息。

Type 2 在数值分解中使用候选集合进行波前内部协作。分析阶段的候选集合并不等于最终实际参与集合。数值分解建立该节点时，根据实际 NFRONT、贡献块规模、累计负载和内存状态确定 NSLAVES 及实际进程表。在强制候选约束的路径中，

$$\{\text{actual slaves}\} \subseteq \{\text{candidate ranks}\};$$

部分内部策略不强制候选约束，此时选择范围可以扩大到 master 之外的更多工作 rank。无论采用哪条路径，实际进程表一经确定，因子块就按该表分布；随后的前代和回代直接复用实际进程表，不再重新寻找空闲 rank[2]。

4.7 PROCNODE_STEPS 的含义

分析阶段把节点顺序转换成数值分解使用的 STEP 顺序，并生成

$$\text{id}\%\text{PROCNODE_STEPS}(\text{step}).$$

该整数不是单纯的 MPI rank。MUMPS_ENCODE_PROCNODE 通过 MUMPS_ENCODE_TPN_IPROC 把节点类型和 0-based owner rank 编码到同一个整数中。使用时，源码分别通过解码函数取得 owner 或节点类型，例如

$$\text{MUMPS_PROCNODE}(\text{PROCNODE_STEPS}(\text{step}), \text{KEEP}(199))$$

返回 owner rank。因而 Fortran 表达式

```
id%PROCNODE_STEPS(step)
```

表示 MUMPS 实例 `id` 中第 `step` 个数值分解步骤的“类型 + owner”编码项；% 是 Fortran 派生类型的成员访问符，括号是数组下标。

4.8 数值分解阶段的 MPI 组织

4.8.1 初始数据分布

集中式输入时，矩阵最初位于 `host`；分布式输入时，各 `rank` 已持有自己的输入项。分析完成后，MUMPS 根据 `PROCNODE_STEPS` 和波前索引把原矩阵项发送到相应 `owner`。分解阶段不要求每个 `rank` 保存完整矩阵；每个 `rank` 保存映射给自己的矩阵项、波前、贡献块和因子。

4.8.2 就绪条件

每个父节点都有尚未完成的依赖计数 `NSTK_STEPS`。一个子节点完成且其结果在父 `owner` 上处理后，源码执行

$$\text{NSTK_STEPS}(\text{STEP}(\text{parent})) \leftarrow \text{NSTK_STEPS}(\text{STEP}(\text{parent})) - 1.$$

当计数变为 0 时，调用 `DMUMPS_INSERT_POOL_N` 把父节点插入本地就绪池。这里的“0”表示该父节点需要的所有子贡献均已处理，不表示同一深度的所有节点均已完成。

4.8.3 异步主循环

`DMUMPS_FAC_PAR` 中，每个工作 `rank` 在未完成任务存在时交替进行：

1. 检查并处理到达的 MPI 消息；
2. 在没有待处理消息时，从本地就绪池中选择节点；
3. 装配和分解该节点；
4. 保存本地因子，把贡献块或控制信息发送给父节点或相关参与进程；
5. 更新本地负载状态和依赖计数。

不同树枝的 Type 0/1 节点可以同时运行。Type 2 节点由 `master` 负责主元和任务控制，候选进程处理其持有的面板或更新数据；因而消去树并行与一个大型波前内部的 MPI 并行可以重叠。

分析映射和运行时调度的边界如下：

- Type 0/1 节点的 `owner` 在分析阶段确定，分解阶段仍在该 `owner` 上处理；
- Type 2 的允许参与集合在分析阶段确定，运行时在该集合内进行动态协作；
- 本地就绪池在多个已映射到本 `rank` 的可执行节点之间选择任务；
- 消息到达时间和实际负载决定节点何时就绪以及候选进程何时参与。

这里的动态性体现在就绪次序、消息推进和 Type 2 实际参与进程的受限选择上；Type 0/1 的 `owner` 关系仍保持分析阶段的映射结果。

4.8.4 Type 0/1: 树间并行与进程内计算

Level-0 子树内部的节点具有同一 owner，子贡献通常直接保留在该 rank 的工作区中；不同 Level-0 子树则可以在不同 rank 上同时推进。一个 rank 可以拥有多棵 Level-0 子树，也可以拥有多个上层 Type 1 节点。

Type 1 节点由唯一 owner 完成装配、主元选择、面板分解、Schur 补更新和因子保存。若父子 owner 不同，子节点将贡献块及其索引发送给父节点 owner。Type 1 的 MPI 并行来自不同 rank 同时处理消去树的不同波前，而不是把单个 Type 1 波前拆分到多个 rank。

4.8.5 Type 2: master/slave 波前内部并行

Type 2 master 在节点真正建立时选择实际 slaves，并记录每个参与进程负责的块行范围。master 负责波前控制、主元选择和主元面板分解；实际 slaves 保存分配给自己的因子或更新块，并接收已分解面板。对第 k 个面板，参与进程并行执行局部更新

$$C_q \leftarrow C_q - L_q U_k,$$

其中 C_q 和 L_q 是 rank q 持有的局部块。master 分解后续面板时，其他 rank 可以继续处理前一面板的更新，从而形成面板分解与尾部更新的流水线。候选进程只有在被选入实际 slave 表后才保存相应数据并参加该节点计算。

4.8.6 Type 3: 二维分布的根波前

进入 Type 3 根之前，子节点贡献块按照根矩阵的二维块循环分布拆分并发送到相应网格进程，而不是全部集中到协调 rank。各网格进程装配自己的局部根矩阵后，共同建立 ScaLAPACK 描述符并执行并行稠密分解。一般非对称根调用 PDGETRF，对称正定根调用 PDPOTRF；未参加根网格的 rank 不持有根矩阵局部块。

三类执行路径构成从树底到树根的并行性转换：树底主要依靠大量独立波前并行，树的中上部依靠 Type 2 波前内部协作，满足条件的根波前使用二维进程网格并行。

4.9 延迟主元和估计偏差对调度的影响

延迟主元使贡献块和祖先波前大于符号估计，因此增加实际浮点运算、通信数据量和进程本地内存。分析阶段已经生成的 owner、Type 2 候选表和树父子关系不会因此整体重做。

若 Type 1 节点因延迟变大，其 owner 不会被普通运行时路径迁移到另一个空闲 rank；该 owner 继续处理实际波前。若 Type 2 节点变大，运行时仍受该节点的实际参与表和所选候选策略约束。强制候选约束时只能从候选表中选择；不强制候选约束的内部路径可以扩大初次选择范围，但并不构成分解过程中的任意工作迁移。

当估计负载与实际负载存在差异时，空闲进程可以执行已经映射到本 rank 的其他就绪子树，在建立 Type 2 节点时被选为实际 slave，或者继续推进通信。此类动态行为不改

变 Type 1 的 owner 关系，也不会求解阶段把未持有因子的空闲 rank 临时加入已有节点。

因此，延迟主元的并行代价包括：父链上的额外计算、较大的贡献块消息、额外装配与内存移动，以及静态估计与实际工作量不一致造成的进程等待。

4.10 求解阶段的 MPI 并行

分解结束后，因子继续分布在生成它们的 rank 上。求解阶段不先把完整 L, U, D 集中到 host，而是沿因子和消去树依赖通信。

4.10.1 前代

一般非对称前代求解

$$Ly = Pb,$$

对称情形还要处理 D 。每个 rank 初始化自己所拥有节点的未完成子依赖计数，并把叶节点加入就绪池。树底不同子树可以同时访问各自保存的因子；一个节点完成局部三角求解和接口变量更新后，将父节点需要的贡献向量发送给父 owner。若父子同属一个 rank，则直接累加到该 rank 的压缩右端项工作区；若 owner 不同，则通过 MPI 发送。父节点在所有子贡献到达后进入就绪池。

Type 2 节点的 master 解出或分发主元相关向量，实际 slaves 使用各自保存的局部因子块并行形成部分更新。通信只传递求解所需的向量和控制信息，不重新传输完整因子。

前代到达 Type 3 根后，右端项按根进程网格的分布方式组织；一般非对称和对称正定路径分别使用 ScaLAPACK 根因子求解例程处理分布式根系统。

4.10.2 回代

回代求解

$$Ux = y \quad \text{或} \quad L^T x = z.$$

依赖方向与分解和前代相反：父节点先得到所需解分量，再把子节点更新所需的数据发送给对应 owner；不同树枝收到父级数据后可以并行展开。普通节点只等待父节点提供接口解，不需要再次等待其全部子节点，因此一个父节点完成后可以同时释放多个子树。

Type 2 回代采用“分发-局部更新-归并”方式。master 向实际 slaves 发送接口解，各 slave 用自己的因子块计算部分右端项更新，master 汇总这些更新后完成主元块回代。这里使用的是分解阶段写入节点因子头的实际 slave 表，而不是分析阶段的 CANDIDATES。

Type 3 根的因子与右端项按二维网格分布，根求解完成后再将所需接口解发送到根以下的节点。一般非对称和对称正定路径分别使用 PDGETRS 和 PDPOTRS。

ICNTL(20) 控制右端项的集中式、稀疏或分布式输入形式，ICNTL(21) 控制解的集中式或分布式返回。多个右端项使节点局部的向量更新变为矩阵更新，并调用相应的 BLAS-3 核。

求解阶段不再进行主元选择，也不会产生新的延迟主元。数值分解中已经发生的延迟会通过实际因子规模、因子所在 rank 和树上的通信量影响求解。

4.11 MPI 进程私有内存与无冲突访问

MPI rank 具有独立虚拟地址空间。不同 rank 上同名的 Fortran 数组不是同一对象，因此两个进程对各自 $A(100)$ 的写入不会发生共享内存冲突。跨进程数据只通过 MPI 发送、接收、广播或集合通信转移。

经典分解路径中，每个 rank 主要维护：

- 实数或复数工作数组 A ：本地波前、因子、贡献块和数值工作区；
- 整数工作数组 IW ：节点头、行列索引、长度、状态和位置描述；
- PTRFAC/PTRAST/PTRIST/PTLUST 等节点位置数组；
- MPI 发送与接收缓冲；
- 本 rank 所需的右端项和解片段。

这些位置量是进程私有 Fortran 数组的下标或偏移，不是跨 MPI rank 有效的共享指针。MUMPS 通常不是为每个节点建立一个独立堆对象，而是在每个 rank 的大工作数组中连续或分区存放多个节点的数据，再通过位置数组定位。

单个 MPI rank 的本地数值工作空间示意

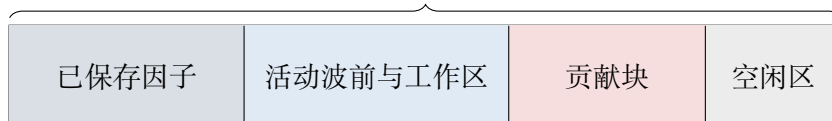


图 4.1 因子、活动数据和贡献块位于各 rank 自己的地址空间。

因子和贡献块具有不同生命周期。节点分解完成后，主元块、耦合因子、置换信息和索引一直保留到求解结束；贡献块只用于父节点装配，完成装配后即可释放或复用。父节点不会把子节点因子并入自己的因子区，而只接收子节点的贡献块。全局 L, U 或 L, D 因子由分布在多个 rank 上的节点因子块及其索引共同表示。

4.11.1 MPI 发送和接收缓冲

跨 rank 的贡献块、面板和求解向量必须经过消息缓冲。非阻塞发送返回后，MPI 可能仍在读取发送数据，因此相应缓冲区在请求完成前不能复用。MUMPS 将已打包消息与 `MPI_Request` 一起记录，并通过 `MPI_TEST` 或等价的完成检查确认发送结束；只有请求完成后，该环形缓冲区块才重新进入空闲状态。

接收端先探测消息类型和长度，再把一条消息接收到进程私有缓冲区并解包。普通事件循环一次处理一条接收消息，因此不会由多个 MPI 接收同时覆盖同一缓冲区。由此，进程间的无冲突性来自两个层次：不同 rank 的地址空间彼此隔离；同一 rank 内尚未完成的非阻塞通信缓冲受请求状态保护。

4.12 贡献块申请、复用与压缩

贡献块具有有限生命周期：子节点完成后生成，在父节点装配后可以释放；数值因子通常保留到求解结束。DMUMPS_ALLOC_CB 根据整数描述长度和贡献块数值长度申请本地空间。若当前连续空闲区不足，内存路径可以执行以下操作：

1. 使用允许的子节点区域或栈区域进行原位放置；
2. 回收已经装配完成、不再需要的贡献块；
3. 移动可移动对象并压缩空洞；
4. 使用动态贡献块存储路径；
5. 在仍无法满足请求时返回内存错误。

IPTRLU/LRLU/IWPOS/IWPOSCB/POSFAC 等变量维护本地工作数组边界、剩余长度和当前因子位置。压缩只修改本 rank 的数组和本地位置记录，不修改其他 MPI rank 的地址。

延迟发生时，实际主元数小于计划主元数，贡献块必须同时保存普通更新变量和延迟变量。DMUMPS_FAC_STACK 根据实际贡献块计算可复用区和移动边界。延迟可能使原位空间不足，从而触发更大的贡献块申请、因子或贡献块移动以及更高的内存高水位。

4.13 内存控制与 out-of-core

主要内存控制参数包括：

- ICNTL(14)：规定数值阶段工作空间相对分析估计的增加百分比；
- ICNTL(23)：规定每个工作进程允许使用的内部工作内存上限，单位为 MB；
- ICNTL(22)：选择 in-core 或 out-of-core 因子存储。

当 ICNTL(23)=0 时，各进程依据分析估计和 ICNTL(14) 分配工作空间；当其为正数时，MUMPS 在给定的每进程上限内安排整数工作区、实数或复数主工作区、通信缓冲和动态存储。ICNTL(14) 的默认值随 MPI 进程数而变化，通常在 20% 至 35% 之间；单进程对称正定情形的默认值为 5%。数值主元导致额外填充时，可以提高该松弛比例 [1]。

out-of-core 模式把完整秩数值因子写入各 rank 管理的磁盘文件；求解阶段在使用相应因子时再读取。活动波前、贡献块、通信缓冲和分解工作区仍需要进程私有内存。分析阶段的内存估计用于决定工作数组和数据分布，数值分解后的统计量则反映延迟主元、实际主元和运行时存储路径共同形成的实际使用量。

同一 MPI rank 内启用 OpenMP 时，各线程共享该 rank 的地址空间。共享统计量和池状态由 MUMPS 的线程区域划分及必要的原子/同步操作维护；多线程 BLAS 内部的数据划分由所链接的 BLAS 实现负责。MPI rank 之间则依靠地址空间隔离保证本地工作数组互不冲突。

第 A 章

延迟主元推广到超节点的两种主元块分解算法

本附录给出延迟主元向超节点法推广时使用的两类稠密主元块算子。非对称算子计算带行、列置换的部分 LU 分解，对称算子计算带对称置换的部分 LDL^T 分解。两类算子的共同目标是：只消去当前超节点中能够形成稳定主元的变量，将其余变量及对应 Schur 补作为扩展贡献块传递给父节点。

A.1 非对称稠密主元块的延迟 LU

A.1.1 波前分块与输出形式

设当前波前的完全求和变量集合为 E ，普通更新变量集合为 U 。按 E, U 排列后，

$$F = \begin{bmatrix} F_{EE} & F_{EU} \\ F_{UE} & F_{UU} \end{bmatrix}.$$

图 A.1 非对称波前中的完全求和块、因子耦合区和贡献区。

若 E 中只有子集 $A \subseteq E$ 能够成功消去，其余集合记为

$$D = E \setminus A,$$

则行、列置换后的部分分解写成

$$P_F F Q_F = \begin{bmatrix} L_{AA} & 0 \\ L_{RA} & I \end{bmatrix} \begin{bmatrix} U_{AA} & U_{AR} \\ 0 & S_R \end{bmatrix}, \quad R = D \cup U.$$

当前节点保存 $L_{AA}, U_{AA}, L_{RA}, U_{AR}$, 并将 S_R 与集合 R 的索引传给父节点。若采用独立行、列超节点结构, 则贡献块必须分别保存行索引和列索引。

A.1.2 步骤一: 分解完全求和块

设第 k 步的活动子块为

$$S^{(k)} = \left[s_{ij}^{(k)} \right]_{i,j=k}^f,$$

其中 $f = |E|$ 。在活动子块的第 k 列中选择绝对值最大的元素:

$$p_k = \arg \max_{k \leq i \leq f} |s_{ik}^{(k)}|, \quad \alpha_k = |s_{p_k k}^{(k)}|.$$

同时记活动子块的最大元素为

$$\beta_k = \|S^{(k)}\|_{\max} = \max_{k \leq i, j \leq f} |s_{ij}^{(k)}|.$$

给定接受阈值 $\tau \in (0, 1)$, 若

$$\alpha_k \geq \tau \beta_k,$$

则采用列主元。用行置换 P_k 将第 p_k 行换到第 k 行:

$$\tilde{S}^{(k)} = P_k S^{(k)}, \quad d_k = \tilde{s}_{kk}^{(k)}.$$

若 $\alpha_k < \tau \beta_k$, 可以在整个活动子块中选取最大元素:

$$(p_k, q_k) = \arg \max_{k \leq i, j \leq f} |s_{ij}^{(k)}|,$$

并用行、列置换将其移动到 (k, k) :

$$\tilde{S}^{(k)} = P_k S^{(k)} Q_k, \quad d_k = s_{p_k q_k}^{(k)}, \quad |d_k| = \beta_k.$$

主元确定后计算

$$(L_k)_i = \frac{\tilde{s}_{ik}^{(k)}}{d_k}, \quad k < i \leq f, \quad (U_k)_j = \tilde{s}_{kj}^{(k)}, \quad k < j \leq f,$$

并执行右看更新

$$s_{ij}^{(k+1)} = \tilde{s}_{ij}^{(k)} - (L_k)_i (U_k)_j, \quad k < i, j \leq f.$$

若 $\beta_k = 0$, 则活动子块中不存在非零主元, 剩余变量不能在当前节点继续消去, 应进入延迟集合。

A.1.3 列延迟策略

全主元搜索可以改写为列移动。若当前活动列在尚未分解的行范围内全为零，则该列在后续右看更新中仍保持为零，因为对应主元行元素为零，更新项

$$L(:, k)U(k, j)$$

也为零。因而可以把该列移到活动区尾部，再从右向左寻找第一个可用非零列：

1. 从活动区末端向前搜索，找到第一个含非零元素的列；
2. 将该列与当前零列交换，并记录列置换；
3. 在换入列中执行列主元搜索和行交换；
4. 若活动区中不存在可用列，则将剩余全部变量延迟。

通过维护活动区终点，可以保证已经放入延迟后缀的零列不会再次交换回候选区，从而避免重复搜索。

A.1.4 分块 KJI-SAXPY 实现

逐主元立即更新整个尾矩阵会产生大量 BLAS-1/2 操作。采用宽度为 b 的面板后，面板内执行主元搜索和细粒度更新，面板外更新合并为 BLAS-3 运算。设一个面板接受了 $t \leq b$ 个主元，记

$$L_P = L(:, k : k + t - 1), \quad U_P = U(k : k + t - 1, :),$$

则尾矩阵统一更新为

$$A_{\text{trail}} \leftarrow A_{\text{trail}} - L_P U_P.$$

面板是“活动行数 $\times b$ ”的长方形列块，主元搜索仍覆盖当前列的全部活动行，而不是局限于对角的 $b \times b$ 方块。面板内准备处理一列时，必须先物化该列尚未写回的更新：

$$A(k : m, k) \leftarrow A(k : m, k) - L(k : m, s : k - 1)U(s : k - 1, k),$$

其中 s 是面板起点。

出现延迟时，当前面板立即结束。算法先将已经接受的 t 个主元对活动区的贡献完整写回，再移动或延迟相应列，然后从新的活动列开始下一面板。这样可以保证后续主元搜索始终作用于当前 Schur 补。

A.1.5 步骤二：更新波前其余部分

分解结束后，将完全求和变量分成已消去集合 A 和延迟集合 D ，并按 A, D, U 重排：

$$F = \begin{bmatrix} F_{AA} & F_{AD} & F_{AU} \\ F_{DA} & F_{DD} & F_{DU} \\ F_{UA} & F_{UD} & F_{UU} \end{bmatrix}.$$

	A	D	U
	$L_{AA} \setminus U_{AA}$	U_{AD}	U_{AU}
	L_{DA}	S_{DD}	S_{DU}
	L_{UA}	S_{UD}	S_{UU}

图 A.2 非对称部分分解后的因子区和扩展贡献块。

先通过三角求解得到与普通更新变量的耦合因子：

$$U_{AU} = L_{AA}^{-1} F_{AU}, \quad L_{UA} = F_{UA} U_{AA}^{-1}.$$

延迟变量与更新变量上的剩余子块统一由 Schur 补给出：

$$\begin{bmatrix} S_{DD} & S_{DU} \\ S_{UD} & S_{UU} \end{bmatrix} = \begin{bmatrix} F_{DD} & F_{DU} \\ F_{UD} & F_{UU} \end{bmatrix} - \begin{bmatrix} L_{DA} \\ L_{UA} \end{bmatrix} \begin{bmatrix} U_{AD} & U_{AU} \end{bmatrix}.$$

展开后，

$$S_{DD} = F_{DD} - L_{DA} U_{AD}, \tag{A.1}$$

$$S_{DU} = F_{DU} - L_{DA} U_{AU}, \tag{A.2}$$

$$S_{UD} = F_{UD} - L_{UA} U_{AD}, \tag{A.3}$$

$$S_{UU} = F_{UU} - L_{UA} U_{AU}. \tag{A.4}$$

A.1.6 步骤三：扩展贡献块传递

子节点传出的扩展贡献块及其索引为

$$C_c = \begin{bmatrix} S_{DD} & S_{DU} \\ S_{UD} & S_{UU} \end{bmatrix}, \quad R_c^{\text{out}} = D_c^r \cup U_c^r, \quad C_c^{\text{out}} = D_c^c \cup U_c^c.$$

数值块中的第 (i, j) 个元素必须与全局行号 $R_c^{\text{out}}(i)$ 和全局列号 $C_c^{\text{out}}(j)$ 绑定。父节点原有候选主元集和更新集分别记为 E_p, U_p ，则装配延迟变量后

$$E_p^{\text{new}} = E_p \cup D_c, \quad I_p^{\text{new}} = E_p^{\text{new}} \cup U_p.$$

父节点建立全局编号到局部位置的映射

$$\text{map}_r(g), \quad \text{map}_c(g),$$

并执行扩展累加

$$F_p(\text{map}_r(R_c^{\text{out}}(i)), \text{map}_c(C_c^{\text{out}}(j))) += C_c(i, j).$$

多个子节点对同一位置的贡献必须累加，不能覆盖。延迟变量进入父节点的完全求和候选区并重新接受主元检验；若仍不能消去，则继续随新的扩展贡献块向祖先节点传递。

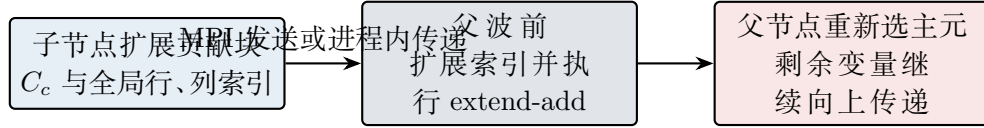


图 A.3 扩展贡献块沿消去树向父节点传播。

A.2 对称稠密主元块的延迟 Bunch–Kaufman 分解

A.2.1 对称波前与分解形式

对称波前满足

$$F = \begin{bmatrix} F_{EE} & F_{EU} \\ F_{EU}^T & F_{UU} \end{bmatrix} = F^T.$$



图 A.4 对称波前中的完全求和块与贡献区。

对称交换必须同时作用于行和列，部分分解形式为

$$P_F^T F P_F = \begin{bmatrix} L_{AA} & 0 \\ L_{RA} & I \end{bmatrix} \begin{bmatrix} D_{AA} & 0 \\ 0 & S_R \end{bmatrix} \begin{bmatrix} L_{AA}^T & L_{RA}^T \\ 0 & I \end{bmatrix}, \quad R = D \cup U,$$

其中 D_{AA} 由 1×1 和 2×2 对角块组成。

A.2.2 Bunch–Kaufman 主元选择

设第 k 步活动对称子块为 $S^{(k)}$ ，定义

$$\gamma = \frac{1 + \sqrt{17}}{8} \approx 0.640388,$$

$$a_k = |s_{kk}^{(k)}|, \quad p = \arg \max_{k < i \leq f} |s_{ik}^{(k)}|, \quad \lambda_k = |s_{pk}^{(k)}|.$$

若 $a_k \geq \gamma \lambda_k$ ，直接采用 $s_{kk}^{(k)}$ 作为 1×1 主元。否则计算候选行的最大非对角元素

$$\sigma_p = \max_{\substack{k \leq j \leq f \\ j \neq p}} |s_{pj}^{(k)}|.$$

随后依次判断：

1. 若

$$a_k \geq \gamma \frac{\lambda_k^2}{\sigma_p},$$

仍采用 $s_{kk}^{(k)}$ 作为 1×1 主元；

2. 若

$$|s_{pp}^{(k)}| \geq \gamma \sigma_p,$$

对称交换变量 k, p ，采用 $s_{pp}^{(k)}$ 作为 1×1 主元；

3. 否则选择变量 k, p 组成 2×2 主元，并通过对称交换把变量 p 移到位置 $k + 1$ 。

对于

$$D_k = \begin{bmatrix} a & b \\ b & c \end{bmatrix}, \quad \Delta_k = ac - b^2,$$

有

$$D_k^{-1} = \frac{1}{\Delta_k} \begin{bmatrix} c & -b \\ -b & a \end{bmatrix}.$$

除 Bunch–Kaufman 选择规则外，还可增加近奇异检查，例如

$$\text{rcond}(D_k) \geq \tau_D$$

或

$$|\Delta_k| > \varepsilon_{\text{piv}} \|D_k\|^2.$$

若候选块不满足稳定性要求，则相应变量进入延迟区，在父节点更大的候选集中重新选取主元。

A.2.3 对称 Schur 补更新

设本步接受的主元阶数为 $r \in \{1, 2\}$ 。对称置换后,

$$\tilde{S}^{(k)} = \begin{bmatrix} D_k & B_k^T \\ B_k & C_k \end{bmatrix}.$$

乘子和 Schur 补为

$$L_k = B_k D_k^{-1}, \quad S^{(k+r)} = C_k - L_k D_k L_k^T = C_k - B_k D_k^{-1} B_k^T.$$

对于 1×1 主元 $D_k = [d_k]$,

$$L_k = \frac{B_k}{d_k}, \quad S^{(k+1)} = C_k - \frac{B_k B_k^T}{d_k}.$$

对于 2×2 主元,

$$L_k = \frac{1}{\Delta_k} B_k \begin{bmatrix} c & -b \\ -b & a \end{bmatrix}, \quad S^{(k+2)} = C_k - L_k D_k L_k^T.$$

每接受一个 1×1 主元, 活动位置前进一列; 每接受一个 2×2 主元, 活动位置前进两列。未通过检验的变量不进入 D_{AA} , 而是通过对称交换移动到延迟后缀。

A.2.4 分块实现与工作矩阵

分块算法维护四个连续区间:

$$\underbrace{[0, b)}_{\text{以前面板已消去}} \quad \underbrace{[b, k)}_{\text{当前面板已接受}} \quad \underbrace{[k, e)}_{\text{活动候选}} \quad \underbrace{[e, n)}_{\text{延迟后缀}}.$$

设当前面板乘子为 L_P , 块对角主元为 D_P , 定义工作矩阵

$$W_P = L_P D_P.$$

在面板贡献尚未整体写回时, 当前 Schur 补表示为

$$S = A_{\text{stored}} - L_P W_P^T.$$

选择主元前, 算法使用矩阵-向量乘法物化所需的当前列; 若判定还需要候选行, 则再物化对应行。显式保存 W_P 可以避免在每次物化时重新遍历 D_P 中混合的 1×1 和 2×2 块。

发生对称主元交换时, 活动矩阵、置换数组、已经形成的 L 行以及 W_P 中的对应行必须同步交换。面板结束后, 用

$$A_{\text{trail}} \leftarrow A_{\text{trail}} - L_P W_P^T$$

一次写回尾矩阵的下三角部分；面板边界不能拆开一个 2×2 主元。

A.2.5 对称扩展贡献块

将未消去变量记为 $R = D \cup U$ ，则对称扩展贡献块为

$$C_c = S_R = F_{RR} - L_{RA} D_{AA} L_{RA}^T.$$

由于矩阵保持对称，只需保存一个变量索引集合

$$I_c^{\text{out}} = D_c \cup U_c$$

以及贡献块的一个三角部分。父节点将 D_c 加入完全求和候选集，通过同一个全局到局部映射同时定位行和列，并执行对称扩展累加。若延迟变量在父节点仍不能形成稳定的 1×1 或 2×2 主元，则继续沿消去树向上传递。

第 B 章

KJI-SAXPY 算法与块分解

B.1 KJI 循环次序

设 $A \in \mathbb{R}^{n \times n}$ 进行原位 LU 分解，暂不写置换。第 k 步先计算乘子

$$l_{ik} = \frac{a_{ik}}{a_{kk}}, \quad i = k + 1, \dots, n,$$

再更新尾矩阵

$$a_{ij} \leftarrow a_{ij} - l_{ik}a_{kj}, \quad i, j = k + 1, \dots, n.$$

KJI 次序把 k 放在最外层、 j 放在中层、 i 放在最内层：

```
for k = 1:n-1
    A(k+1:n,k) = A(k+1:n,k) / A(k,k)
    for j = k+1:n
        A(k+1:n,j) = A(k+1:n,j)
            - A(k,j) * A(k+1:n,k)
    end
end
```

固定 k, j 后，内层更新为

$$A(k+1:n, j) \leftarrow A(k+1:n, j) - A(k, j)A(k+1:n, k),$$

即向量形式

$$y \leftarrow y + \alpha x,$$

对应 BLAS-1 的 SAXPY；双精度实数对应 DAXPY。列主序存储下，内层 i 遍历同一列的连续元素。

对每个 k ，全部后续列的更新还可以写成秩一形式

$$A_{k+1:n, k+1:n} \leftarrow A_{k+1:n, k+1:n} - A_{k+1:n, k} A_{k, k+1:n}.$$

该形式对应 BLAS-2 的 GER。SAXPY 和 GER 表示的是同一消元更新的两种接口组织。将多个秩一更新合并为块更新，是稠密分块 LU 和超节点稀疏分解使用 BLAS-3 的基础 [8, 7]。

B.2 多个秩一更新的合并

若连续处理 b 个主元 $k = s, \dots, s + b - 1$, 这些主元对后续子矩阵的总更新为

$$\sum_{q=s}^{s+b-1} l_q u_q^T.$$

把列向量 l_q 和行向量 u_q^T 分别组合成

$$L_P = \begin{bmatrix} l_s & l_{s+1} & \cdots & l_{s+b-1} \end{bmatrix},$$

$$U_P = \begin{bmatrix} u_s^T \\ u_{s+1}^T \\ \vdots \\ u_{s+b-1}^T \end{bmatrix},$$

则

$$\sum_{q=s}^{s+b-1} l_q u_q^T = L_P U_P.$$

因此 b 次秩一更新可以合并成一次矩阵乘法:

$$A_{\text{trail}} \leftarrow A_{\text{trail}} - L_P U_P.$$

块分解并不改变 LU 的代数结果; 它把重复的向量更新重排为矩阵-矩阵运算。

B.3 方形分块 LU

把当前活动矩阵按宽度 b 分块:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad A_{11} \in \mathbb{R}^{b \times b}.$$

块 LU 恒等式为

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & S_{22} \end{bmatrix}.$$

比较分块乘积得到:

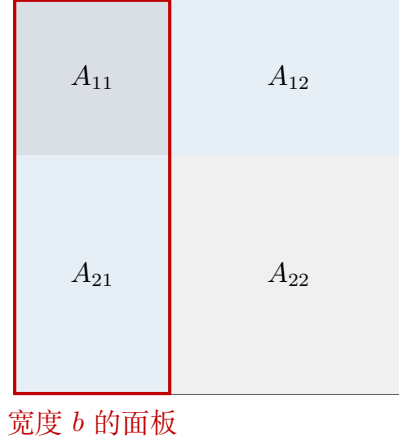


图 B.1 块 LU 的方形活动矩阵和面板。

$$A_{11} = L_{11}U_{11}, \quad (\text{B.1})$$

$$U_{12} = L_{11}^{-1}A_{12}, \quad (\text{B.2})$$

$$L_{21} = A_{21}U_{11}^{-1}, \quad (\text{B.3})$$

$$S_{22} = A_{22} - L_{21}U_{12}. \quad (\text{B.4})$$

对应一次面板迭代的计算顺序为：

1. 在面板中分解主元块 A_{11} ；
2. 用左三角求解计算 U_{12} ；
3. 用右三角求解计算 L_{21} ；
4. 用矩阵乘法更新 A_{22} 。

第 2、3 步对应 TRSM，第 4 步对应 GEMM。标量 KJI 中连续的 DAXPY/GER 更新由最后一步的 GEMM 集中完成。

B.4 主元交换与面板更新

采用部分选主元时，第 k 列的主元位置为

$$p_k = \arg \max_{i \geq k} |a_{ik}|.$$

搜索范围是当前列的全部活动行，不限于 $b \times b$ 的对角方块。因此，算法中的“面板”是高为活动行数、宽为 b 的长方形列块；figure B.1 中红色区域表示这一列块。

行交换必须同步作用于当前活动矩阵和已经形成的 L 行，并记录到行置换中。若算法还允许列交换，则活动矩阵的列、全局列编号和列置换必须同步更新，分解关系写成

$$PAQ = LU.$$

面板内部不能直接使用尚未包含前面板列贡献的旧数据。设当前面板起点为 s ，已经接受 t 个主元，准备处理 $k = s + t$ 时，当前列先物化为

$$A(k:n, k) \leftarrow A(k:n, k) - L(k:n, s:k-1)U(s:k-1, k).$$

面板结束后，再把全部已接受列对面板外尾矩阵的贡献一次写回。

B.5 延迟主元下的分块 LU

非对称延迟 LU 算子采用 64 列面板。面板内每一步仍先检查当前列；只有当前列全零时，才按算子规定的活动子块或活动列搜索规则继续选取主元。

若搜索发生在面板尚未完成时，必须先落实该面板已经接受的主元对搜索区域的更新。设面板已经接受 $t \leq b$ 列，则写回

$$A_{\text{trail}} \leftarrow A_{\text{trail}} - L_P(:, 1:t)U_P(1:t, :).$$

写回后，被检查的活动区是当前 Schur 补。若仍不存在可用主元，则以当前位置为 `info`，剩余区间作为延迟后缀。

因此，固定面板宽度 b 是一次最多收集的主元数；遇到延迟时，实际面板宽度可以小于 b 。

B.6 对称块更新

对称不定分解中，一个面板收集 1×1 和 2×2 主元块。设面板乘子为 L_P ，块对角主元为 D_P ，尾矩阵更新为

$$A_{22} \leftarrow A_{22} - L_P D_P L_P^T.$$

定义

$$W_P = L_P D_P,$$

则

$$A_{22} \leftarrow A_{22} - L_P W_P^T.$$

分块 Bunch-Kaufman 算子用

$$S = A_{\text{stored}} - L_P W_P^T$$

表示面板中尚未写回的 Schur 补。主元测试需要某一行或某一列时，以矩阵-向量乘法计算相应部分；面板完成后，以对称秩二 k 更新写回下三角尾矩阵。面板边界不允许把一个 2×2 主元拆成两部分。

B.7 块三角求解

块化同样用于因子求解。把下三角系统分块：

$$\begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix} \begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} = \begin{bmatrix} B_1 \\ B_2 \end{bmatrix}.$$

前代为

$$L_{11}Y_1 = B_1,$$

$$B'_2 = B_2 - L_{21}Y_1,$$

$$L_{22}Y_2 = B'_2.$$

上三角回代同理。单个右端项时，中间更新是矩阵-向量乘法；多个右端项组成矩阵 B 时，更新

$$B'_2 = B_2 - L_{21}Y_1$$

成为矩阵-矩阵乘法。因而，块因子格式既用于分解阶段合并 SAXPY 更新，也用于求解阶段合并多个右端项的三角更新。

B.8 运算量与数据访问

无论采用标量 KJI 还是块 LU，稠密 $n \times n$ LU 的主导浮点运算量均约为

$$\frac{2}{3}n^3.$$

二者的区别是运算组织。标量形式反复读取乘子列和后续列并执行向量更新；块形式把 b 个主元的更新合并，使 L_P 和 U_P 的元素在一次矩阵乘法中重复使用。面板内部仍保留主元搜索、交换和细粒度更新，面板外的主要 Schur 补更新则由 TRSM 和 GEMM 完成。

这一变换可以概括为

$$\underbrace{\sum_{q=s}^{s+b-1} l_q u_q^T}_{b \text{ 次秩一更新}} \longrightarrow \underbrace{L_P U_P}_{\text{一次块更新}}.$$

参考资料

- [1] MUMPS Consortium. *MUltifrontal Massively Parallel Solver (MUMPS 5.6.2): Users' Guide*, 2023.
- [2] MUMPS Consortium. *MUMPS 5.6.2 Source Distribution*, 2023.
- [3] I. S. Duff and J. K. Reid. The multifrontal solution of indefinite sparse symmetric linear equations. *ACM Transactions on Mathematical Software*, 9(3):302–325, 1983.
- [4] P. R. Amestoy, I. S. Duff, J.-Y. L'Excellent, and J. Koster. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.
- [5] P. R. Amestoy, A. Guermouche, J.-Y. L'Excellent, and S. Pralet. Hybrid scheduling for the parallel solution of linear systems. *Parallel Computing*, 32(2):136–156, 2006.
- [6] J. W. H. Liu. The multifrontal method for sparse matrix solution: theory and practice. *SIAM Review*, 34(1):82–109, 1992.
- [7] J. W. Demmel, S. C. Eisenstat, J. R. Gilbert, X. S. Li, and J. W. H. Liu. A supernodal approach to sparse partial pivoting. *SIAM Journal on Matrix Analysis and Applications*, 20(3):720–755, 1999.
- [8] T. A. Davis. *Direct Methods for Sparse Linear Systems*. SIAM, 2006.
- [9] J. R. Bunch and L. Kaufman. Some stable methods for calculating inertia and solving symmetric linear systems. *Mathematics of Computation*, 31(137):163–179, 1977.